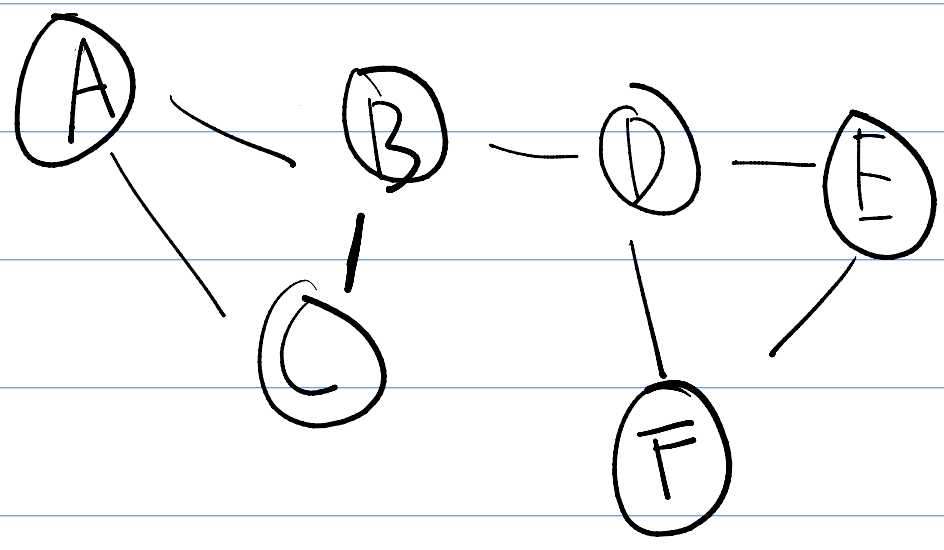


Network - Ex 04

4A1 1. $C_d(v) = \frac{\text{deg}(v)}{(|V|-1)}$ $\rightarrow$ total node - 1



nodes	deg	C-d
A	2	2/5
B	3	3/5
C	2	2/5
D	3	3/5
E	2	2/5
F	2	2/5

Betweenness Centrality
Nodes in which lies on the most shortest paths between other nodes

- 2. By inspection B and D, the nodes has 2 clusters A-B-C and D-E-F and it is connected in the middle by B-D
- 3. degree = B and D = 3/5
betweenness = also B and D which makes them the same nodes
- 4 Structural role of B and D is that it acts as a bridge connecting the two clusters (A,B,C) and (D,E,F). Removing B or D would disconnects the 2 groups.

4.A.2

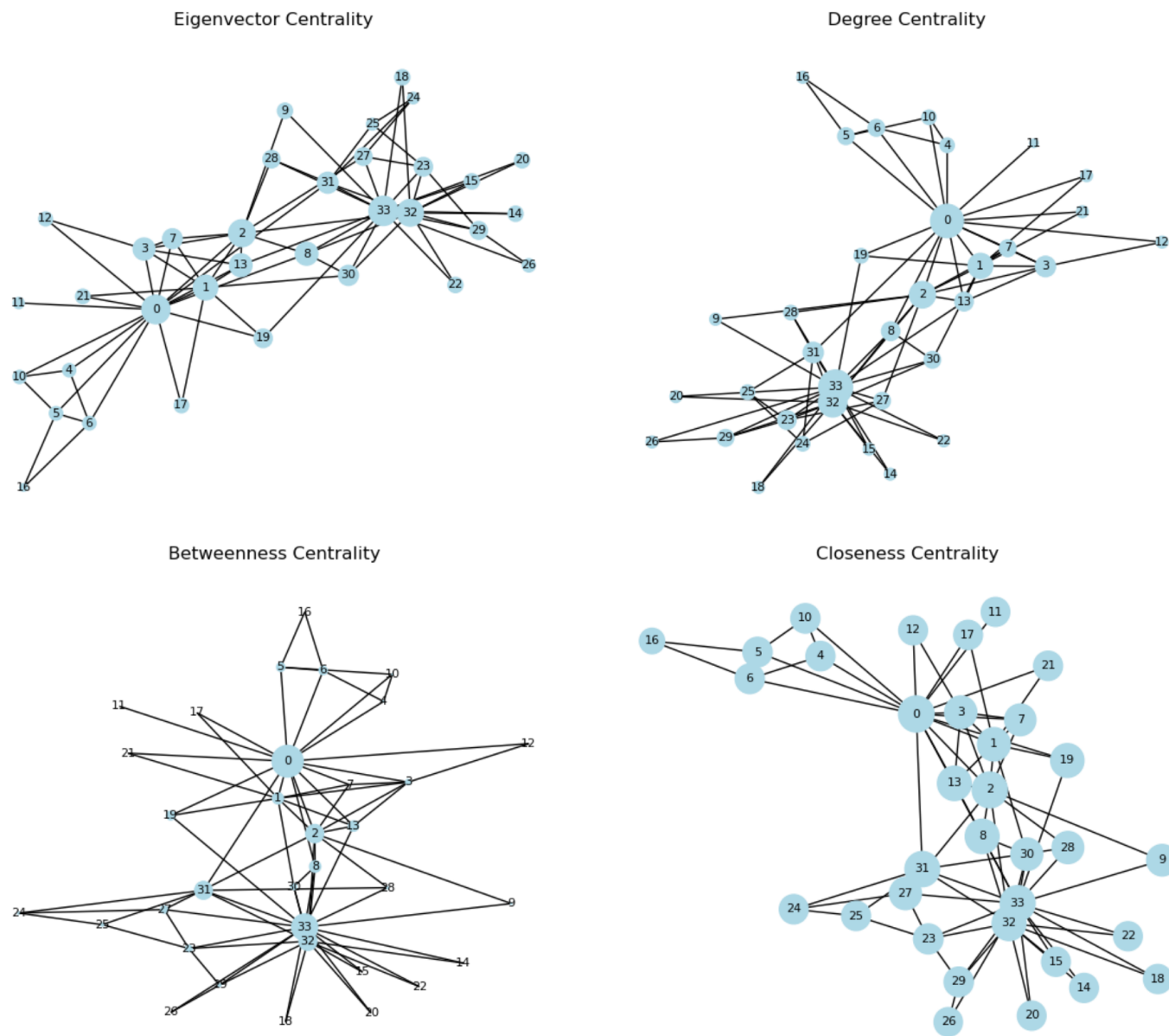
```
study > snippets > network-ex04 > 4.A.2.py > ...
1 import networkx as nx
2 import matplotlib.pyplot as plt
3
4 G = nx.karate_club_graph()
5
6 #centralities
7 centralities = [nx.eigenvector_centrality(G), nx.degree_centrality(G), nx.betweenness_centrality(G), nx.closeness_centrality(G)]
8 print("Centralities:")
9 print(f"Eigenvector Centrality : {centralities[0]}")
10 print(f"Degree Centrality : {centralities[1]}")
11 print(f"Betweenness Centrality : {centralities[2]}")
12 print(f"Closeness Centrality : {centralities[3]}")
13
14 #top-3 nodes for each centrality
15 top_nodes = [sorted(centrality.items(), key=lambda x: x[1], reverse=True)[:3] for centrality in centralities]
16 print("\nTop-3 nodes for each centrality:")
17 centrality_names = ["Eigenvector Centrality", "Degree Centrality", "Betweenness Centrality", "Closeness Centrality"]
18 for name, nodes in zip(centrality_names, top_nodes):
19     print(f"{name}: {nodes}")
20
21 #plot four subplots, each showing the network with node sizes proportional to one centrality measure.
22 plt.figure(figsize=(12, 10))
23 for i, (centrality, name) in enumerate(zip(centralities, centrality_names)):
24     plt.subplot(2, 2, i + 1)
25     node_sizes = [v * 1000 for v in centrality.values()]
26     nx.draw(G, with_labels=True, node_size=node_sizes, node_color='lightblue', font_size=8)
27     plt.title(name)
28 plt.tight_layout()
29 plt.show()
30
31 #Find a node that ranks very differently across measures
32 print("\nNodes with varying rankings across centrality measures:")
33 for node in G.nodes():
34     eigen_rank = sorted(centralities[0].items(), key=lambda x: x[1], reverse=True).index((node, centralities[0][node])) + 1
35     degree_rank = sorted(centralities[1].items(), key=lambda x: x[1], reverse=True).index((node, centralities[1][node])) + 1
36     betweenness_rank = sorted(centralities[2].items(), key=lambda x: x[1], reverse=True).index((node, centralities[2][node])) + 1
37     closeness_rank = sorted(centralities[3].items(), key=lambda x: x[1], reverse=True).index((node, centralities[3][node])) + 1
38     rank_diff = max(eigen_rank, degree_rank, betweenness_rank, closeness_rank) - min(eigen_rank, degree_rank, betweenness_rank, closeness_rank)
39     if rank_diff >= 3:
40         print(f"Node {node}: Eigen={eigen_rank}, Degree={degree_rank}, Betweenness={betweenness_rank}, Closeness={closeness_rank} (Diff={rank_diff})")
```

5959595, 25: 0.0038404882154882154, 26: 0.0, 27: 0.02233345358345358, 28: 0.0017947330447330447, 29: 0.0029220779220779218
Closeness Centrality : {0: 0.5689655172413793, 1: 0.4852941176470588, 2: 0.559322033898305, 3: 0.4647887323943662, 4: 0.3703793103448275862, 11: 0.36666666666666664, 12: 0.3707865168539326, 13: 0.515625, 14: 0.3707865168539326, 15: 0.3707865168539326, 23: 0.39285714285714285, 24: 0.375, 25: 0.375, 26: 0.3626373626373626, 27: 0.4583333333333333, 28: 0.55}

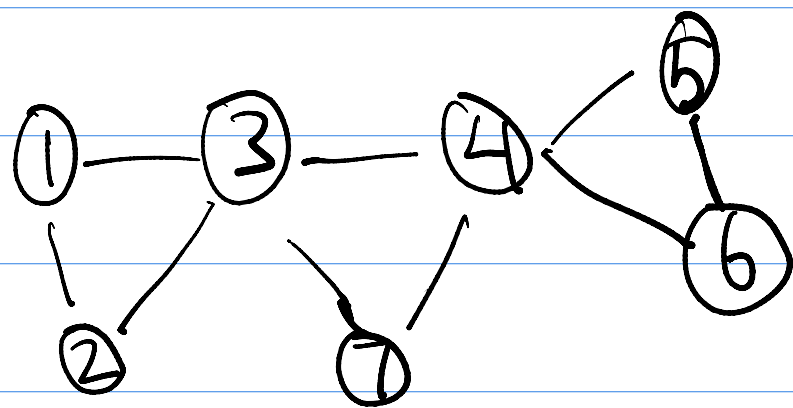
Top-3 nodes for each centrality:
Eigenvector Centrality: [(33, 0.37337121301323506), (0, 0.35548349418519426), (2, 0.3171893899684447)]
Degree Centrality: [(33, 0.5151515151515151), (0, 0.48484848484848486), (32, 0.36363636363636365)]
Betweenness Centrality: [(0, 0.43763528138528146), (33, 0.30407497594997596), (32, 0.145247113997114)]
Closeness Centrality: [(0, 0.5689655172413793), (2, 0.559322033898305), (33, 0.55)]

Nodes with varying rankings across centrality measures:
Node 1: Eigen=5, Degree=5, Betweenness=7, Closeness=9 (Diff=4)
Node 3: Eigen=8, Degree=6, Betweenness=15, Closeness=10 (Diff=9)
Node 4: Eigen=28, Degree=17, Betweenness=21, Closeness=20 (Diff=11)
Node 5: Eigen=26, Degree=11, Betweenness=10, Closeness=17 (Diff=16)

Node 5: Eigen=26, Degree=11, Betweenness=10, Closeness=17 (Diff=16)
Node 6: Eigen=27, Degree=12, Betweenness=11, Closeness=18 (Diff=16)
Node 7: Eigen=11, Degree=13, Betweenness=23, Closeness=14 (Diff=12)
Node 8: Eigen=6, Degree=8, Betweenness=6, Closeness=5 (Diff=3)
Node 9: Eigen=17, Degree=23, Betweenness=20, Closeness=15 (Diff=8)
Node 10: Eigen=29, Degree=18, Betweenness=22, Closeness=21 (Diff=11)
Node 11: Eigen=33, Degree=34, Betweenness=24, Closeness=32 (Diff=10)



4B1.



embedded

its removal barely affects connectivity

broker

its removal disconnects clusters

- ① Node 1 = embedded Since the 2 triangle clusters are connected by node 3-4
- ② Node 7 = broker. Since node 7 connects 3 and 4. the bridge between the 2 clusters. Its C_7 is 1
- ③ C_7 is now 0 now it becomes the only node connecting the 2 clusters. It became a critical bridge.
- ④ Node 7 can learn information from cluster {4,5,6}

4.B.2 ③. because its two neighbors cluster are tied and it provides an alternative path between these 2 large clusters

```

import networkx as nx
import matplotlib.pyplot as plt

G = nx.Graph()
G.add_edges_from([(1, 2), (1, 3), (2, 3), (4, 5), (4, 6), (5, 6), (3, 7), (4, 7)])

# betweenness centrality of all nodes
bc = nx.betweenness_centrality(G)
print("Betweenness centrality:")
for node, score in bc.items():
    print(f"{node}: {score:.3f}")

# Visualize with node sizes proportional to betweenness
plt.figure(figsize=(8, 6))
pos = nx.spring_layout(G, seed=42)
node_sizes = [v * 3000 for v in bc.values()]
nx.draw_networkx(G, pos, with_labels=True, node_size=node_sizes, node_color='skyblue', edge_color='gray')
plt.title("Network with Node Sizes Proportional to Betweenness Centrality")
plt.show()

```

```

G = nx.karate_club_graph()
# sort nodes top 3 by betweenness centrality (highest) but lowest clustering coefficient first
bc = nx.betweenness_centrality(G)
cc = nx.clustering(G)
sorted_nodes = sorted(G.nodes(), key=lambda n: (bc[n], -cc[n]), reverse=True)[:3]
print("\nNodes sorted by betweenness centrality (highest first) and clustering coefficient:")
for node in sorted_nodes:
    print(f"Node {node}: Betweenness={bc[node]:.3f}, Clustering={cc[node]:.3f}")

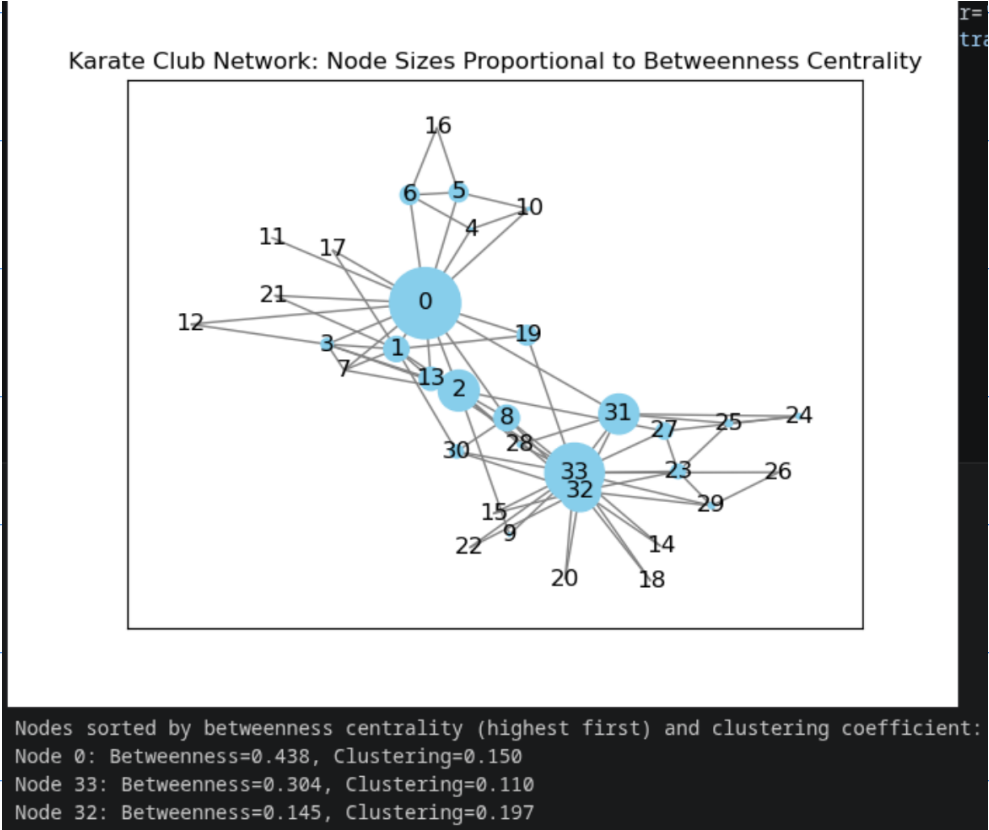
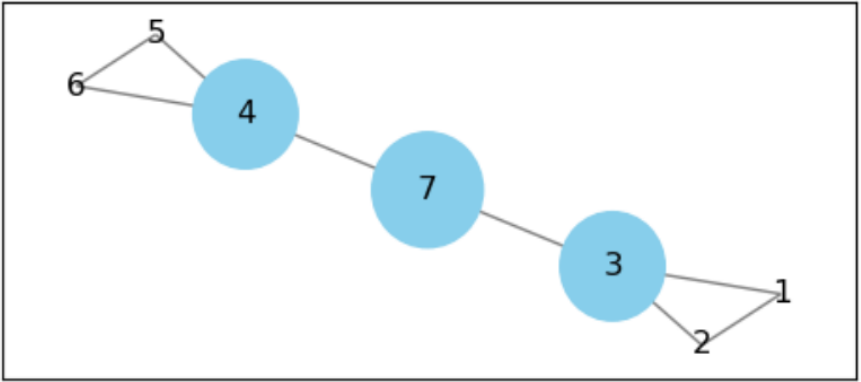
```



```
# Visualize the Karate Club graph with node sizes proportional to betweenness centrality
plt.figure(figsize=(10, 8))
pos = nx.spring_layout(G, seed=42)
node_sizes = [bc[n] * 3000 for n in G.nodes()]
nx.draw_networkx(G, pos, with_labels=True, node_size=node_sizes, node_color='skyblue', edge_color='gray')
plt.title("Karate Club Network: Node Sizes Proportional to Betweenness Centrality")
plt.show()
```

```
qt.qpa.wayland.textinput
> python3 4.B.2.py
Betweenness centrality:
1: 0.000
2: 0.000
3: 0.533
4: 0.533
5: 0.000
6: 0.000
7: 0.600
█
```

Network with Node Sizes Proportional to Betweenness Centrality



```
Nodes sorted by betweenness centrality (highest first) and clustering coefficient:
Node 0: Betweenness=0.438, Clustering=0.150
Node 33: Betweenness=0.304, Clustering=0.110
Node 32: Betweenness=0.145, Clustering=0.197
```

low clustering and high betweenness means that these nodes are pure bridges. connecting 2 or more clusters.